

ESP32-S3-ETH

From Waveshare Wiki

Jump to: navigation, search

Overview

Usage Instructions

Components Preparation

- ESP32-S3-ETH x1
- PoE Module (B) x1
- OV5640 camera x1
- 32GB TF card x 1
- USB cable (Type-A male to Type-C male) x1

[[Expand](#)]

The ETH board



(<https://www.waveshare.com/esp32-s3-eth.htm?sku=28972>)

ESP32-S3, I2C/UART/USB

The ETH board + OV5640 camera



(<https://www.waveshare.com/esp32-s3-eth.htm?sku=28770>)

ESP32-S3, I2C/UART/USB

The ETH board + POE module



(<https://www.waveshare.com/esp32-s3-eth.htm?sku=28771>)

ESP32-S3, I2C/UART/USB

**The ETH board + POE module +
OV5640 camera**

(<https://www.waveshare.com/esp32-s3-eth.htm?sku=28772>)

ESP32-S3, I2C/UART/USB



(/wiki/File:600px-

ESP32-S3-ETH-Test-prepare.jpg)

Before operating, it is recommended to browse the table of contents to quickly understand the document structure. For smooth operation, please read the FAQ carefully to understand possible problems in advance. All resources in the document are provided with hyperlinks for easy download.

Working with Arduino

This chapter introduces setting up the Arduino environment, including the Arduino IDE, management of ESP32 boards, installation of related libraries, program compilation and downloading, as well as testing demos. It aims to help users master the development board and facilitate secondary development.

Environment Setup

Download and Install Arduino IDE

1. Click to visit the Arduino official website (<https://www.arduino.cc/en/software>), select the corresponding system and system bit to download.



Arduino IDE 2.3.3

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger.

For more details, please refer to the [Arduino IDE 2.0 documentation](#).

Nightly builds with the latest bugfixes are available through the section below.

SOURCE CODE

The Arduino IDE 2.0 is open source and its source code is hosted on [GitHub](#).

DOWNLOAD OPTIONS

Windows Win 10 and newer, 64 bits
Windows MSI installer
Windows ZIP file

Linux AppImage 64 bits (X86-64)
Linux ZIP file 64 bits (X86-64)

macOS Intel, 10.15: "Catalina" or newer, 64 bits
macOS Apple Silicon, 11: "Big Sur" or newer, 64 bits

[Release Notes](#)

(/wiki/File:Arduino-IDE-Download.jpg)

2. Run the installer and install all by default.

For more instructions on how to use the Arduino IDE, please refer to Arduino Official Documentation (<https://docs.arduino.cc/software/ide/>)

Install ESP32 Development Board

- To use ESP32 boards in Arduino IDE, first install the **ESP32 Development Board** package.
- In some areas, it may not be able to **Install online** due to network reason, and **Install offline** is generally recommended.
- For the tutorial on installing **ESP32 Development Board** package, please refer to Arduino board manager tutorial (https://www.waveshare.com/wiki/Arduino_Board_Managers_Tutorial)

- ESP32-S3-ETH Development board installation instructions

Board name	Board installation requirement	Version number requirement
ESP32-S3-ETH	"Install Offline" / "Install Online"	2.0.12 and above

Install Libraries

- When installing Arduino libraries, there are usually two ways to choose from: **Install online** and **Install offline**. For most libraries, users can easily search and install them through the online library manager of the Arduino software. However, some open-source libraries or custom libraries are not synchronized to the Arduino Library Manager, so they cannot be acquired through online searches. In this case, users can only manually install these libraries offline.
- For library installation tutorial, please refer to Arduino library manager tutorial (https://www.waveshare.com/wiki/Arduino_Library_Manager_Tutorial)

- ESP32-S3-ETH library file description

Library Name	Description	Library Installation Requirement
Adafruit_NeoPixel	NeoPixel Light Bar Control Library	"Install Online" or "Install Offline"
ESP32-BLE-Keyboard	ESP32 Bluetooth Keyboard Library	"Install Online" or "Install Offline"
ETHClass	ESP32 Ethernet Library	Install Offline

Run the First Arduino Demo

If you are just getting started with ESP32 and Arduino, and you don't know how to create, compile, [\[Expand\]](#)

flash, and run Arduino ESP32 programs, then please expand and take a look. Hope it can help you!

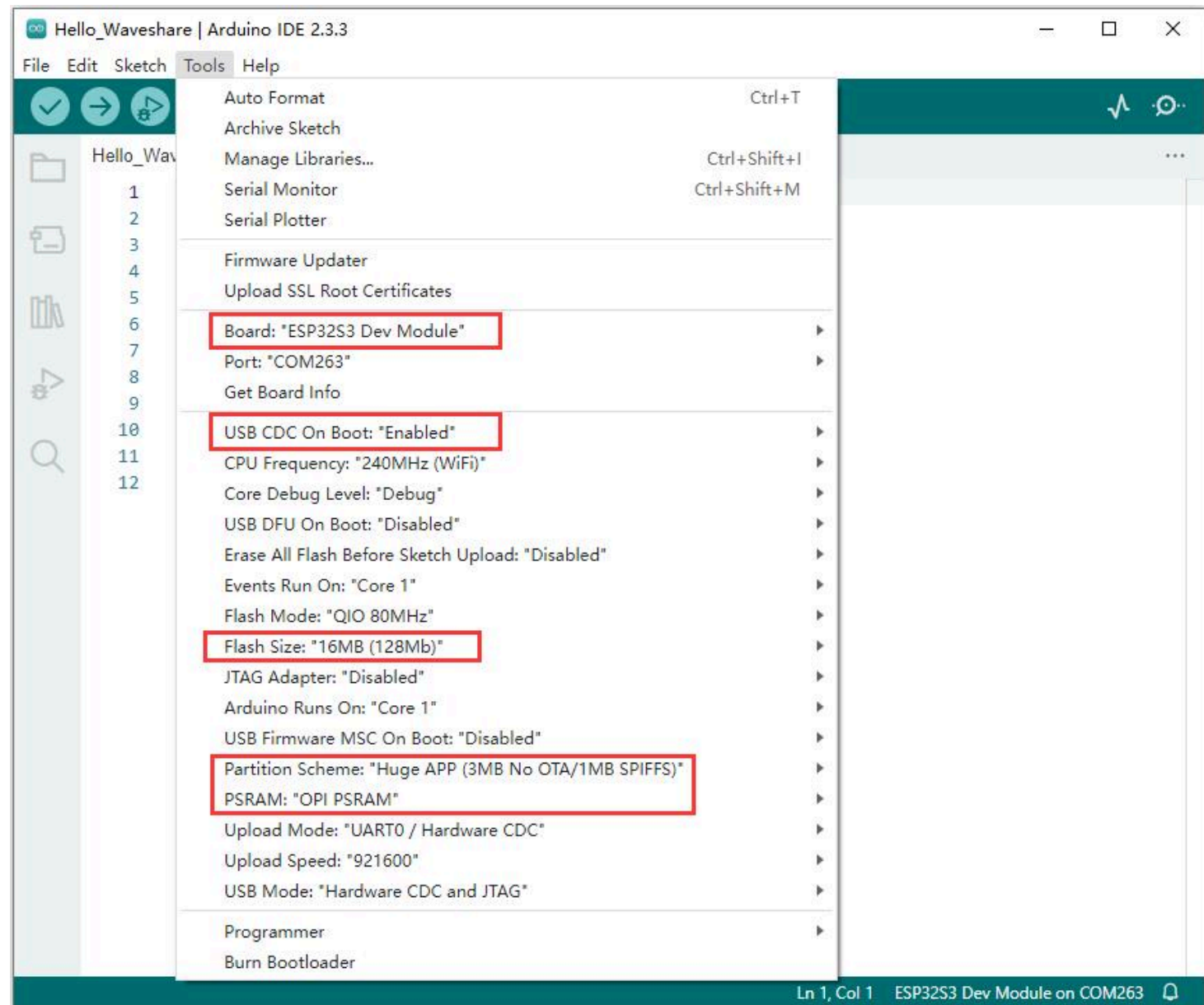
Demo

- ESP32-S3-ETH demos

No.	Demo	Description	Dependency Library
1	IO_Test	Basic demo: GPIO pin high and low level control	-
2	RGB_LED	Basic demo: GPIO pin controls WS2812 LED	Adafruit NeoPixel
3	SD_Card	Basic demo: TF card loading, read and write operations	-
4	BLE_Keyboard	Basic demo: Bluetooth keyboard function demonstration	ESP32-BLE-Keyboard
5	WiFi_AP	Basic demo: Set as an AP hotspot to allow other WiFi devices to connect to the Internet	
6	WiFi_STA	Basic demo: Set to STA mode to access WiFi routing network for networking, and can control the level output of the GPIO port in real time	
7	WiFi_DHCP	Basic demo: Connect via WiFi, assign IP address via DHCP	
8	WiFi_staticIP	Basic demo: Connect via WiFi, statically assign IP address	
9	ETH_DHCP	Basic demo: Connect via Ethernet, assign IP address via DHCP	

10	ETH_staticIP	Basic demo: Connect via Ethernet, statically assign IP address	
11	ETH_MAC_DHCP_StaticIP	Basic demo: Connect via Ethernet, customize the MAC address, use DHCP to automatically obtain IP by default, if the acquisition fails, use the default static IP address directly	
12	WIFI_Web_CAM	Comprehensive demo: Connect via WiFi, implement network camera function	
13	ETH_Web_CAM	Comprehensive demo: Connect via Ethernet, implement network camera function	

- Arduino project configuration:



(/wiki/File:ESP32-S3-ETH-Arduino-Demo-Setting3.jpg)

IO_Test

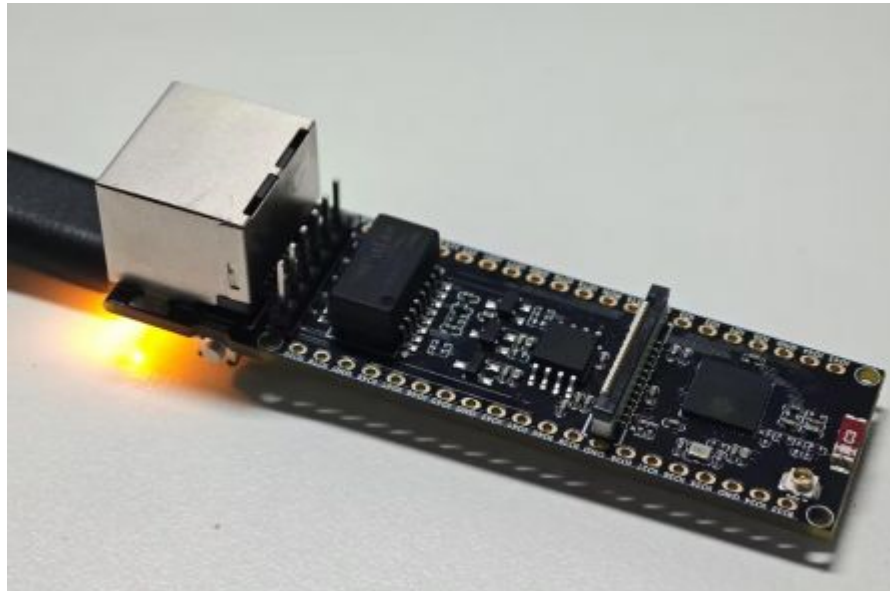
[\[Collapse\]](#)

Demo description

- This demo demonstrates how to use multiple GPIO pins of the ESP32-S3-ETH module as output control ports. The demo sequentially sets each GPIO pin to high (open) and low (closed) in order, with an interval of 300 milliseconds for each state change. In this way, the sequential switching operation of the GPIO pins can be observed.

Hardware connection

- Connect the board to the computer using a USB cable



(/wiki/File:ESP32-S3-ETH-HW-

Connection-IO-TEST.jpg)

Code analysis

1. GPIO pin configuration: The demo first defines the 25 GPIO pins and initializes them into output mode in the `setup()` function. All pins are set to low level (off state) at the beginning.

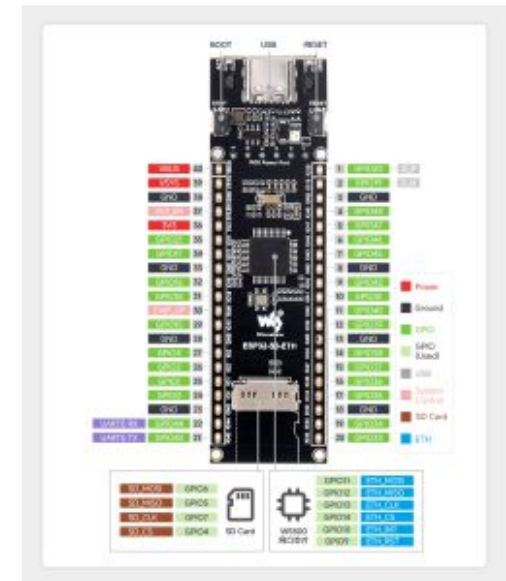
```
#define NUM_GPIO 25

// Define GPIO pins
const int gpio_pin[NUM_GPIO] = {21, 17, 16, 18, 15,
3, 2, 1, 0, 44, 43, 33, 34, 35, 36, 37, 38, 39, 40,
41, 42, 45, 46, 47, 48};
```

2. GPIO control loop:

- In the `loop()` function, the demo will turn on each GPIO pin in turn (set to high) for a delay of 300 milliseconds, and then turn off the pin (set to low).
- This process will be repeated constantly.

3. Print Output: The demo outputs the status information of each GPIO pin through the USB port, indicating whether the pin is in a high voltage level (on) or a low voltage level (off) state.



(/wiki/File:ESP32-S3-ETH-details-15.jpg)

ESP32-S3-ETH GPIO Pinout definition

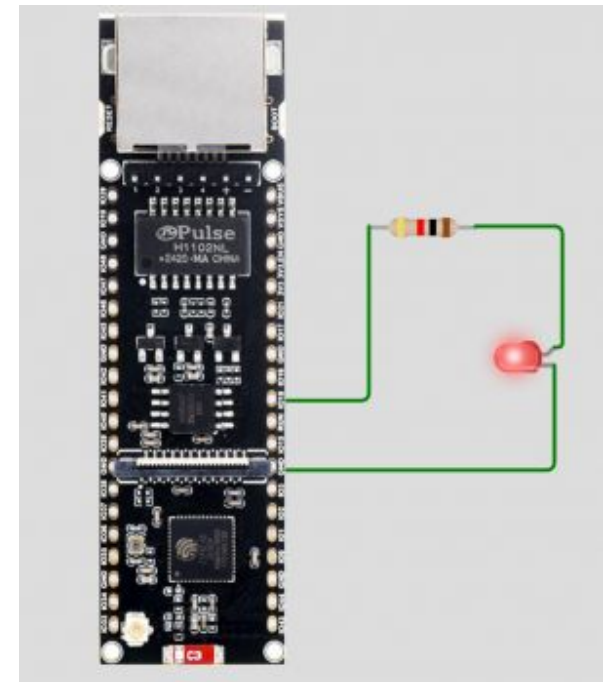
```
void loop() {  
    // Turn on each GPIO one by one  
    for (current_gpio = 0; current_gpio < NUM_GPIO; current_gpio++) {  
        digitalWrite(gpio_pin[current_gpio], HIGH); // Set GPIO pin to HIGH (turn on)  
        printf("GPIO %d set to HIGH.\n", gpio_pin[current_gpio]); // Print GPIO state  
        delay(300); // Delay for 300ms  
    }  
  
    // Turn off each GPIO one by one  
    for (current_gpio = 0; current_gpio < NUM_GPIO; current_gpio++) {  
        digitalWrite(gpio_pin[current_gpio], LOW); // Set GPIO pin to LOW (turn off)  
        printf("GPIO %d set to LOW.\n", gpio_pin[current_gpio]); // Print GPIO state  
        delay(300); // Delay for 300ms  
    }  
}
```

How to know more about the Arduino ESP32 libraries which explain the use of GPIO? Please refer to Arduino ESP32 library GPIO use (<https://docs.espressif.com/projects/arduino-esp32/en/latest/api/gpio.html>).

Result demonstration

After the demo is flashed, the running result of the device is as follows:

- **ESP32-S3-ETH** will control 25 GPIO pins in turn. The demo switches the HIGH and LOW states on each GPIO pin and outputs the state changes to the monitoring window via USB.
The low level of the output is 0V, and the high level is the operating voltage of the current board. Since the operating voltage of ESP32-S3 is 3.3V, the high level is 3.3V.
- Open the serial port monitoring window of the Arduino IDE, and you can observe the state switching of each GPIO pin, as shown in the following figure:



(/wiki/File:ESP32-S3-ETH-HW-IO-LED-TEST.jpg)

GPIO waterfall light experiment diagram

The screenshot shows the Arduino IDE 2.3.3 interface. The top menu bar includes File, Edit, Sketch, Tools, and Help. The toolbar shows icons for checking, uploading, and downloading. The 'ESP32S3 Dev Module' is selected in the board manager. The file 'IO_Test.ino' is open, displaying a C++ sketch with a multi-line comment block. The comment block includes metadata like @file, @author, @version, @date, @brief, @license, and @copyright, followed by an experiment objective, hardware resources, and pin assignment details. The Serial Monitor at the bottom shows the output of the sketch, which is a sequence of GPIO pin status messages.

```

1  /**
2  *
3  * @file    IO_Test.ino
4  * @author  Kaloha
5  * @version V1.0
6  * @date    2024-10-10
7  * @brief   GPIO control experiment
8  * @license MIT
9  * @copyright Copyright (c) 2024, Waveshare
10 *
11 *
12 * Experiment Objective: Learn how to use GPIO as an output to control multiple GPIO pins.
13 *
14 * Hardware Resources and Pin Assignment:
15 * 1. GPIO Pins --> ESP32-S3-ETH Module
16 *   GPIO Pins used: {21, 17, 16, 18, 15, 3, 2, 1, 0, 44, 43, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 45, 46, 47, 48}
17 *
18 * Experiment Phenomenon:
19 * 1. GPIO pins will be set HIGH and LOW sequentially every 300ms.
20 *
21 * Notes:
22 * None
23 *
24 *

```

Serial Monitor Output:

```

GPIO 40 set to HIGH.
GPIO 47 set to HIGH.
GPIO 48 set to HIGH.
GPIO 21 set to LOW.
GPIO 17 set to LOW.
GPIO 16 set to LOW.
GPIO 18 set to LOW.
GPIO 15 set to LOW.

```

(/wiki/File:ESP32-S3-ETH-IO-TEST.jpg)

Usage Scenario: Applicable for learning and testing GPIO pin control on ESP32-S3. Users can use a multimeter to detect the voltage level of each GPIO pin, or perform a simple waterfall light experiment by connecting LEDs and resistors.

RGB_LED

【Demo description】

The demo uses the GPIO21 pin to control a WS2812 RGB LED to display a variety of lighting effects, including Color Wipe, Rainbow, and Theater Chase.

【Hardware connection】

Connect the board to the computer using a USB cable

【Code analysis】

1. GPIO pin configuration: The demo first defines the RGB LED control pin as a GPIO pin and performs the following steps in the `**`setup` function**`:

- Initialize the NeoPixel RGB LED:
 - Use ``strip.begin()`` to enable LED control functionality.
 - Set the LED brightness to 50% using ``strip.setBrightness(50)``.
 - Call ``strip.show()`` to initialize the LED to an off state.
 - Configure an additional indicator LED (LED1) as an output and turn it on using ``digitalWrite(LED1, HIGH)``.


```
#define PIN 21          // Pin connected to WS2812 RGB LED
#define LED1 -1         // GPIO for an additional LED (optional)

Adafruit_NeoPixel strip = Adafruit_NeoPixel(60, PIN, NEO_GRB + NEO_KHZ800);

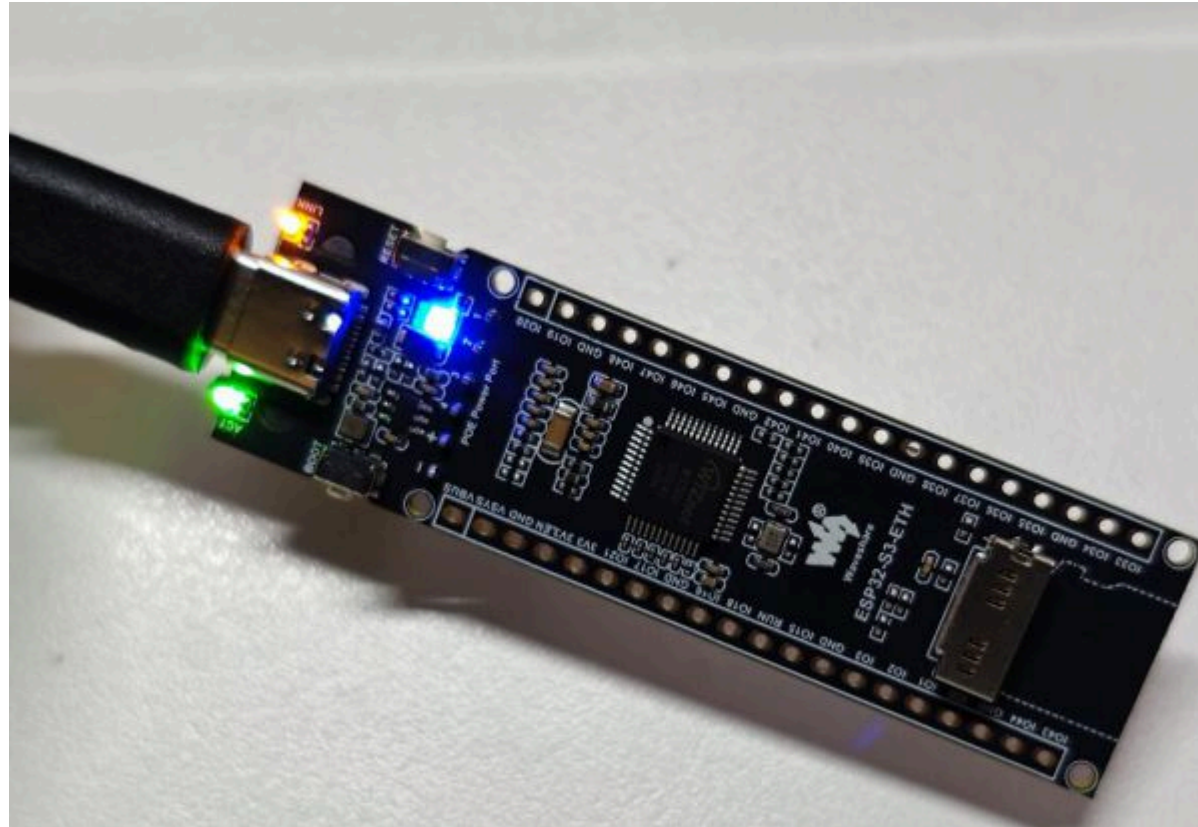
void setup() {
  #if defined (__AVR_ATtiny85__)
    if (F_CPU == 16000000) clock_prescale_set(clock_div_1);
  #endif

  strip.begin();
  strip.setBrightness(50);
  strip.show();

  pinMode(LED1, OUTPUT);
  digitalWrite(LED1, HIGH); // Turn on the additional LED
}
```

【Result demonstration】

- **Color Wipe:** The RGB LED sequentially displays red, green, and blue colors, with each color lasting for a certain period
- **Theater Chase:** Dynamic chasing effects in white, red, and blue, with LEDs displaying colors in a periodic flashing manner



(/wiki/File:ESP32-S3-ETH-RGB-LED.jpg)

SD_Card

[Collapse]

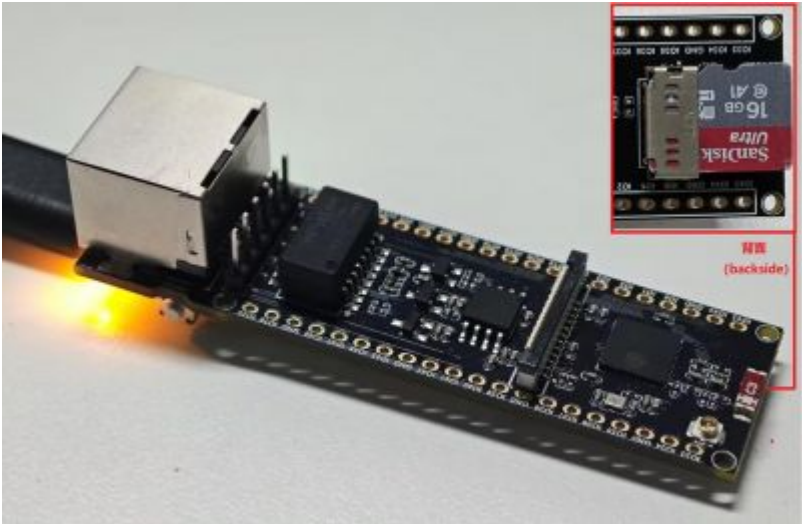
Demo description

- This demo demonstrates how to operate a TF card on the **ESP32-S3-ETH module**, including **initializing the TF card, creating and writing files, listing files in the directory, and reading file content**. The demo

will create a file named "waveshare.txt", write the text "Hello world from Waveshare", list all the file information in the root directory of the TF card, and finally read the contents of the "waveshare.txt" file and print it to the serial monitor.

Hardware connection

- Install a TF card on the board (a TF card below 16GB is recommended) and connect the board to the computer using a USB cable



(/wiki/File:400px-ESP32-S3-ETH-HW-SD_Card.jpg)

TF card control pin description

SPI interface	ESP32-S3-ETH
CS (SS)	GPIO4
DI (MOSI)	GPIO6
DO (MISO)	GPIO5
SCK (SCLK)	GPIO7

Code analysis

1. setup()

- Initialize serial port communication and the TF card.
- Initialize the SPI bus of ESP32 via `SPI.begin()` and specify the pins used for TF card communication.

- Call `SD.begin()` to try to mount the TF card, and output card size upon successful connection.
- Call three core functions:
 - `writeFileToSD()` : Create and write to a file.
 - `listFilesOnSD()` : Scan and print all files in the root directory of the TF card.
 - `readFileFromSD()` : Opens and reads the contents of the `waveshare.txt` file.

2. `writeFileToSD()`

- The main function is to create or open `waveshare.txt` files and write the text "Hello world from Waveshare".
- After the file write is completed, the file is closed to ensure that the write operation completes correctly.

3. `listFilesOnSD()`

- Scan all files in the root directory of the TF card and output the file names through the serial port.
- Use the `openNextFile()` function to open each file in the directory in turn and print their names, and finally close the root directory file.

4. `readFileFromSD(const char *filePath)`

- Open and read the file with the specified path, here is `waveshare.txt`.
- If the file exists, use `file.read()` to read the content byte by byte, output the content of the file through the serial port, and finally close the file.

How to know more about the Arduino ESP32 libraries which explain the use of TF card? Please refer to Arduino ESP32 library TF card use (<https://github.com/espressif/arduino-esp32/tree/master/libraries/SD>).

Result demonstration

After the demo is flashed, the running result of the device is as follows:

- Mount the TF card first, then create and write files, scan and list the root directory files of the TF card, and finally open and read the content of a specific file.

The screenshot shows the Arduino IDE 2.3.3 interface. The top menu bar includes File, Edit, Sketch, Tools, and Help. The toolbar shows icons for checking, running, and uploading code. The board is set to ESP32S3 Dev Module. The sketch file is SD_Card.ino. The code in the editor includes comments about the file name, author (Kaloha), version (V1.1), date (2024-10-24), brief description (SD card read/write and directory scan experiment), license (MIT), and copyright (© 2024, Waveshare). It also includes an experiment objective and hardware resources (SD Card Interface connected to ESP32-S3-ETH Module with pins MISO: 5, MOSI: 6, SCLK: 7, CS: 4). The experiment phenomenon is described as: 1. The SD card will be initialized, and if successful, a file named "waveshare.txt" will be created. 2. The file will contain the text "Hello world from Waveshare". 3. The root directory of the SD card will be scanned, and all files found will be printed to the serial console. 4. The "waveshare.txt" file will be opened, its contents read and printed to the serial console.

The Serial Monitor at the bottom shows the following output:

```

SDCard MOUNT SUCCESS
SDCard Size: 15193 MB
File written successfully
SDCard MOUNT SUCCESS
SDCard Size: 15193 MB
File written successfully
Files in root directory:
System Volume Information
NIHA0.TXT
FOO.TXT
HELLO.TXT
waveshare.txt
Reading from /waveshare.txt:
Hello world from Waveshare

```

The status bar at the bottom indicates the cursor is at line 11, column 4, and the board is connected to COM263 at 115200 baud.

(/wiki/File:800px-ESP32-S3-ETH-SD_Card-Running.jpg)

Usage Scenario: Applicable for learning and demonstrating how to perform file read and write operations on ESP32, or for storing data records such as sensor data, log information, and other persistent storage needs.

BLE_Keyboard

For details, see Open Source demo: ESP32-BLE-Keyboard (<https://github.com/T-vK/ESP32-BLE-Keyboard/tree/master>)

WIFI_AP

[\[Collapse\]](#)

Demo description

- This demo configures the **ESP32-S3-ETH** as a **WiFi access point (AP mode)**, allowing other devices to connect to this WiFi network. The **MAC addresses** of the connected devices and their **assigned IP addresses** will be printed on the serial port monitor. This demo allows you to learn how to use the ESP32 as an access point and dynamically monitor the information of the devices connected to the access point.

Hardware connection

- Connect the board to the computer using a USB cable

Code analysis

1. setup()

- Initialize serial communication and configure ESP32 as a WiFi access point (AP).
- Set up the AP using `WiFi.softAP()` and specify the network name (SSID) as "ESP32-S3-ETH" and the password as "88888888".
- Use `WiFi.onEvent()` to register a WiFi event handler that handles client connections and disconnections.

- Print a confirmation message on the serial port to indicate that the AP initialization was successful.

2. **formatMacAddress()**

- Convert the MAC address from a byte array format to a readable string format (such as `XX:XX:XX:XX:XX:XX`) .
- Used to display the MAC address of the connected device on the serial port.

3. **WiFiEvent()**

- Handle WiFi events, such as client connections and disconnections.
- When the device is connected:
 - Print the MAC address of the device.
 - Add a short delay (500 ms) to ensure DHCP has time to assign IP addresses.
 - Call `printDeviceIP()` to get and display the assigned IP address.
- When the device is disconnected, print the disconnection information.

4. **printDeviceIP(const uint8_t* mac)**

- Retrieves and prints the IP address of the connected device based on the MAC address.
- Use the ESP-IDF functions `esp_wifi_ap_get_sta_list()` and `tcpip_adapter_get_sta_list()` to get a list of connected devices.
- Traverse through the list of devices, compare MAC addresses to identify target devices.
- If the IP address is valid (not `0.0.0.0`), the IP address is printed. Otherwise, it means that DHCP has not assigned an IP address.

5. **loop()**

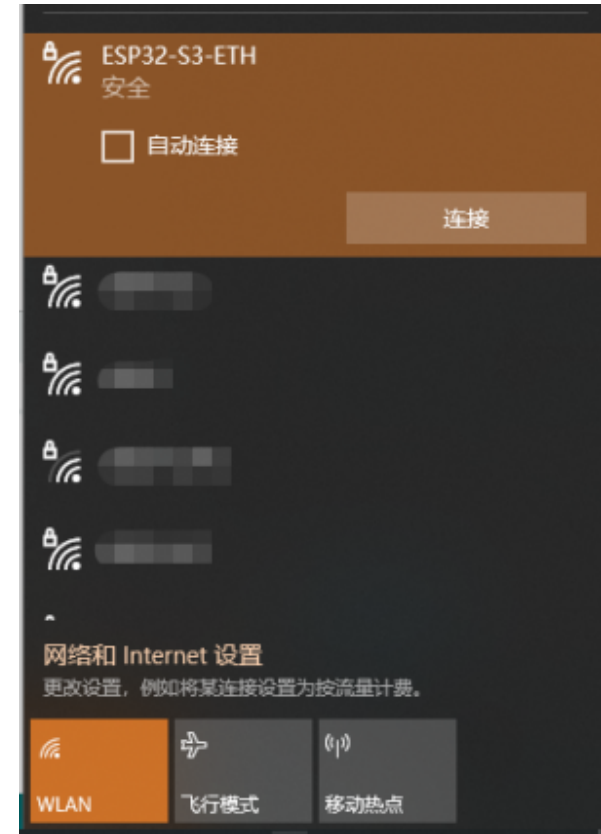
- A small delay is included to keep the CPU idle and prevent unnecessary load.
- There are no other operations in the main loop.

How to know more about the WiFi features of the Arduino ESP32 library? Please refer to Arduino ESP32 library WiFi use (<https://github.com/espressif/arduino-esp32/tree/master/libraries/WiFi>) °

Result demonstration

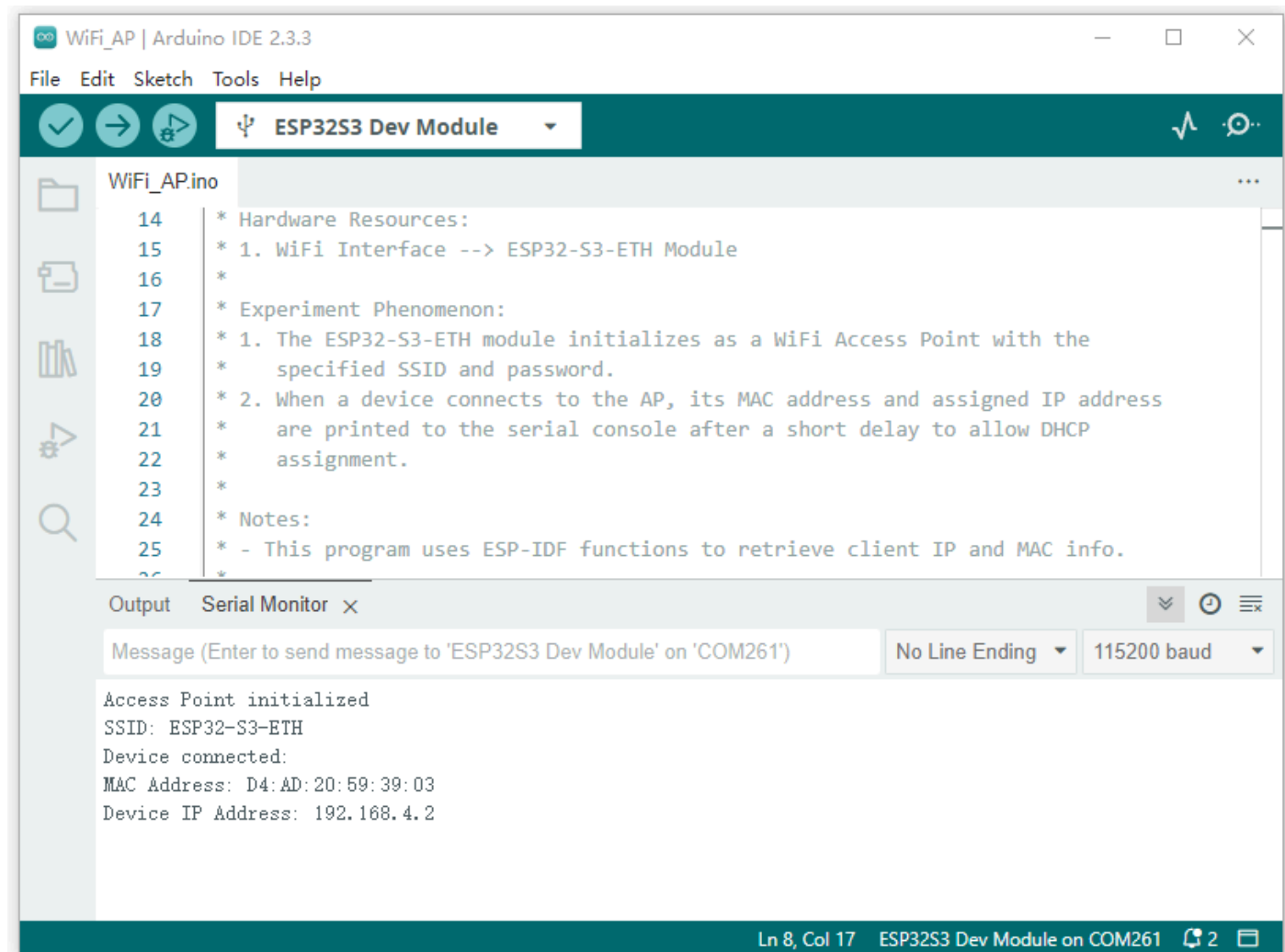
After the demo is flashed, the running result of the device is as follows:

1. The ESP32-S3-ETH module will start a WiFi access point with the network name "ESP32-S3-ETH".
2. When the device is connected to the access point, its MAC address and assigned IP address are printed to the serial monitor.
3. When the device is disconnected, a disconnection message is also output on the serial port.



(/wiki/File:300px-ESP32-S3-ETH-WIFI-AP-Connect.png)

ESP32-S3-ETH WiFi hotspot



(/wiki/File:ESP32-S3-ETH-WIFI-AP-Running.png)

Usage Scenario: Applicable for the creation of local networks and device management in IoT applications. Set the ESP32 as the central node through the access point mode, and monitor the IP and MAC information of the connected devices.

WIFI_STA

[Collapse]

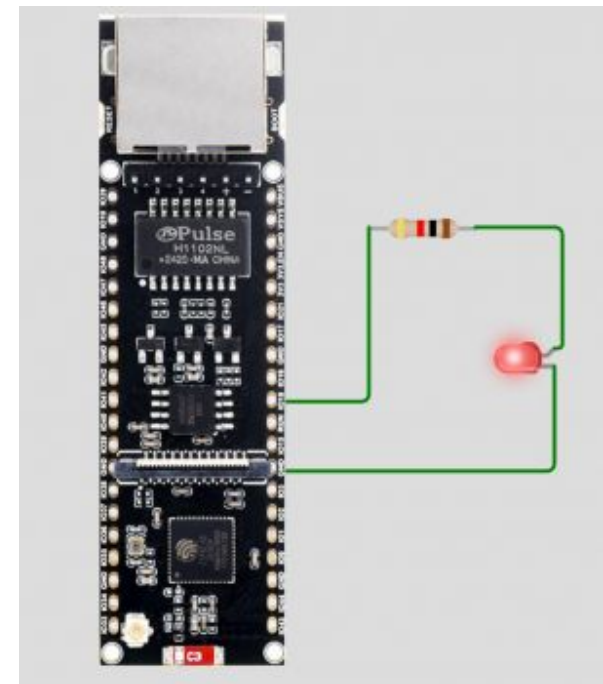
Demo description

- The demo configures the ESP32-S3 module as a WiFi workstation (STA mode) and sets up a web server to control the LEDs connected to the GPIO18 via a web page. The user can check the current LED status (LED ON or LED OFF) on the web page and toggle the switching status of the LED. Each time the LED status changes, the status information is displayed on the web page and on the serial monitor.

Hardware connection

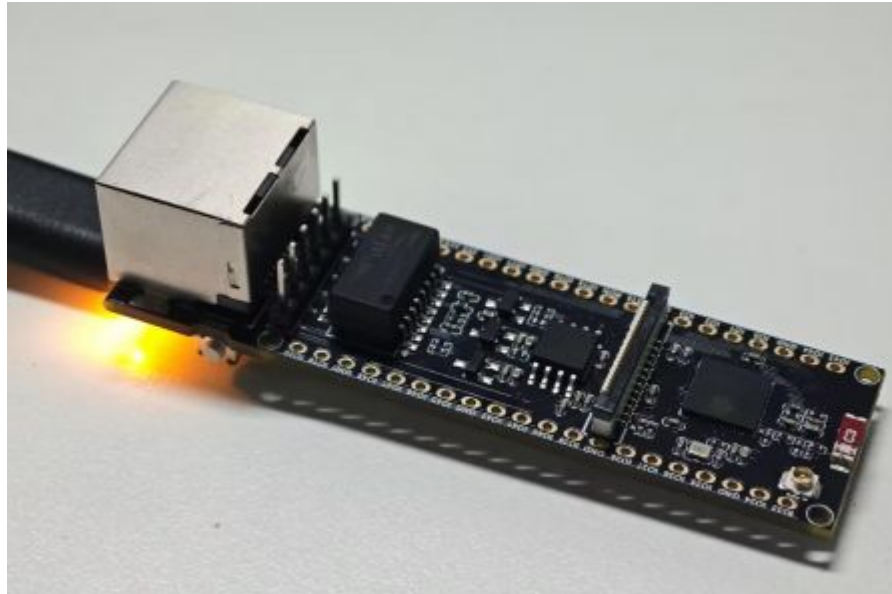
- Connect the board to the computer using a USB cable

If you have the conditions, you can refer to the LED light on the string on the right to measure and check the high and low level status of GPIO18, or measure it with a multimeter.



(/wiki/File:ESP32-S3-ETH-HW-IO-LED-TEST.jpg)

WIFI controls GPIO18 output high and low levels



(/wiki/File:ESP32-S3-ETH-HW-

Connection-IO-TEST.jpg)

Code analysis

1. setup()

- **Function:** Initializes system settings, including serial communication, GPIO configuration, WiFi connection, and web server startup.
- **Main steps:**
 - Initialize serial port communication (for debugging output).
 - Set GPIO18 as the output and make sure the LED is initially off.
 - Connect to a WiFi network and print the IP address on the serial monitor.
 - Start the web server and wait for the client to connect.

2. loop()

- **Function:** Handles client connections and HTTP requests, switches the LED status based on the request content, and generates an HTML page to display the current LED status.

■ Main steps:

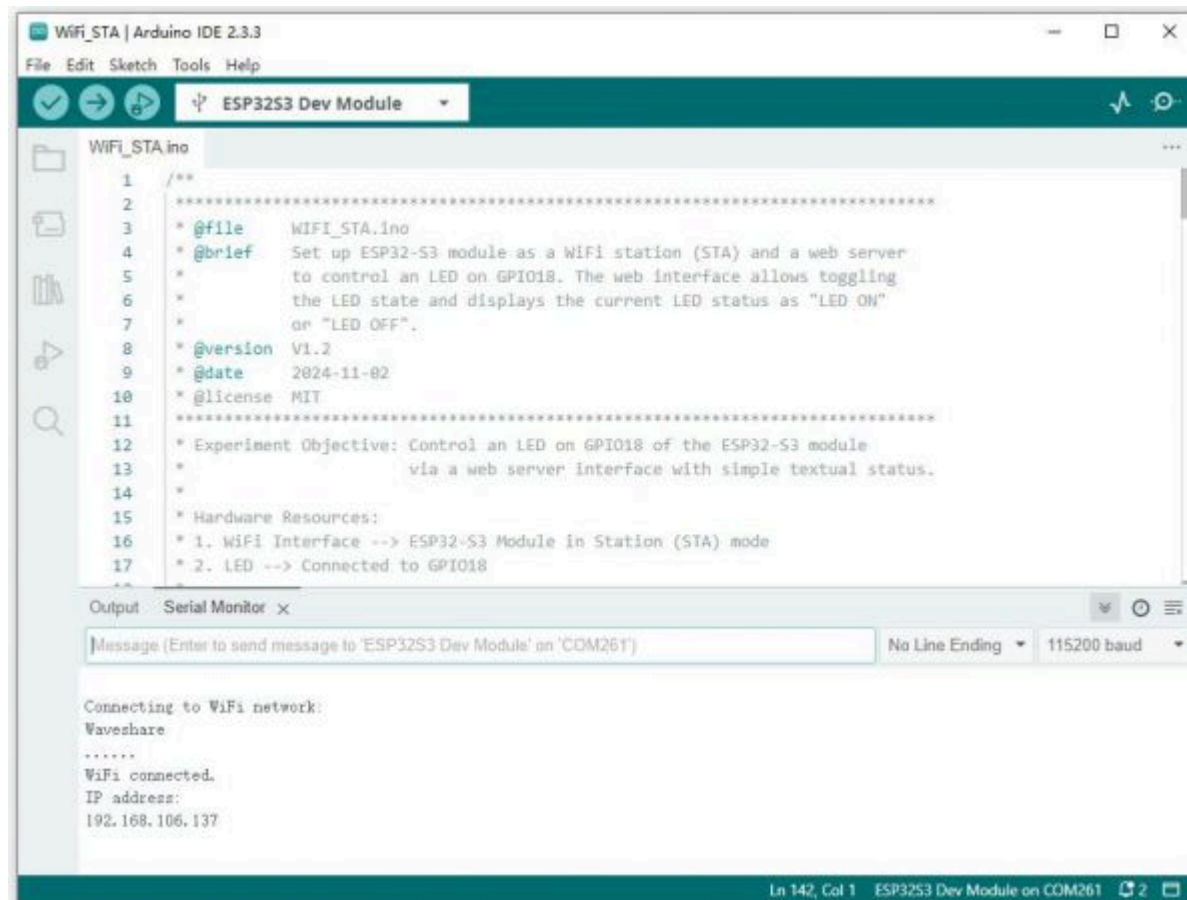
- Check whether a new client is connected and read the HTTP request.
- Control the high or low level of GPIO18 based on the request path "/H" or "/L" to switch the LED status.
- Generate an HTML page containing the current LED status and switch links, then return it to the client.
- Close the connection with the client, wait for the next connection request.

How to know more about the WiFi features of the Arduino ESP32 library? Please refer to Arduino ESP32 library WiFi use (<https://github.com/espressif/arduino-esp32/tree/master/libraries/WiFi>) °

Result demonstration

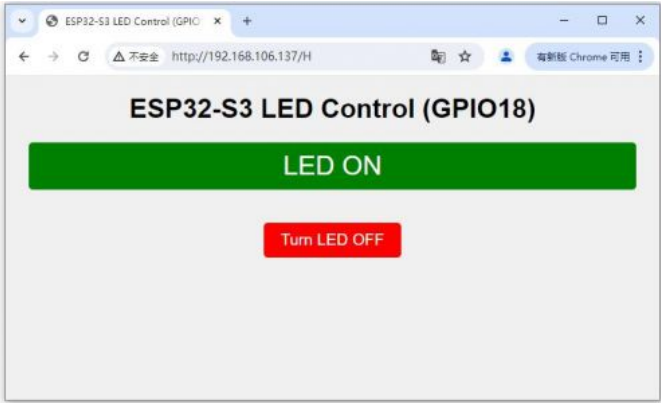
After the demo is flashed, the running result of the device is as follows:

1. After the module is connected to the network, the IP address of the module is printed out in the monitoring window



(/wiki/File:600px-ESP32-S3-ETH-WIFI_STA-running.jpg)

2. Open a web browser and enter the corresponding IP, you can control the high and low level status of the GPIO18 pin in real time, and the corresponding serial port monitoring window will also print out the return status value of H or L in real time.



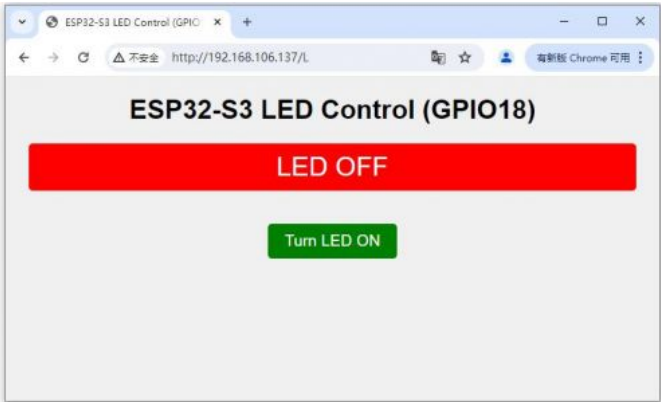
(/wiki/File:497px-ESP32-S3-ETH-WIFI_STA-HIGH-01.jpg)

GPIO18 is set high



(/wiki/File:400px-ESP32-S3-ETH-WIFI_STA-HIGH-02.jpg)

Return: /H



(/wiki/File:497px-ESP32-S3-ETH-WIFI_STA-LOW-01.jpg)

GPIO18 is set low



(/wiki/File:400px-ESP32-S3-ETH-WIFI_STA-LOW-02.jpg)

Return: /L

WIFI_DHCP

[Collapse]

Demo description

The demo connects via WIFI using the **ESP32-S3-ETH module** and assigns an IP address through DHCP.

Code analysis

1. **WIFI network configuration:** You need to change the WIFI and password in the code to the name and password in the WIFI router in your environment. If there is none, you can create a hotspot with the same name using your phone for module connection testing.

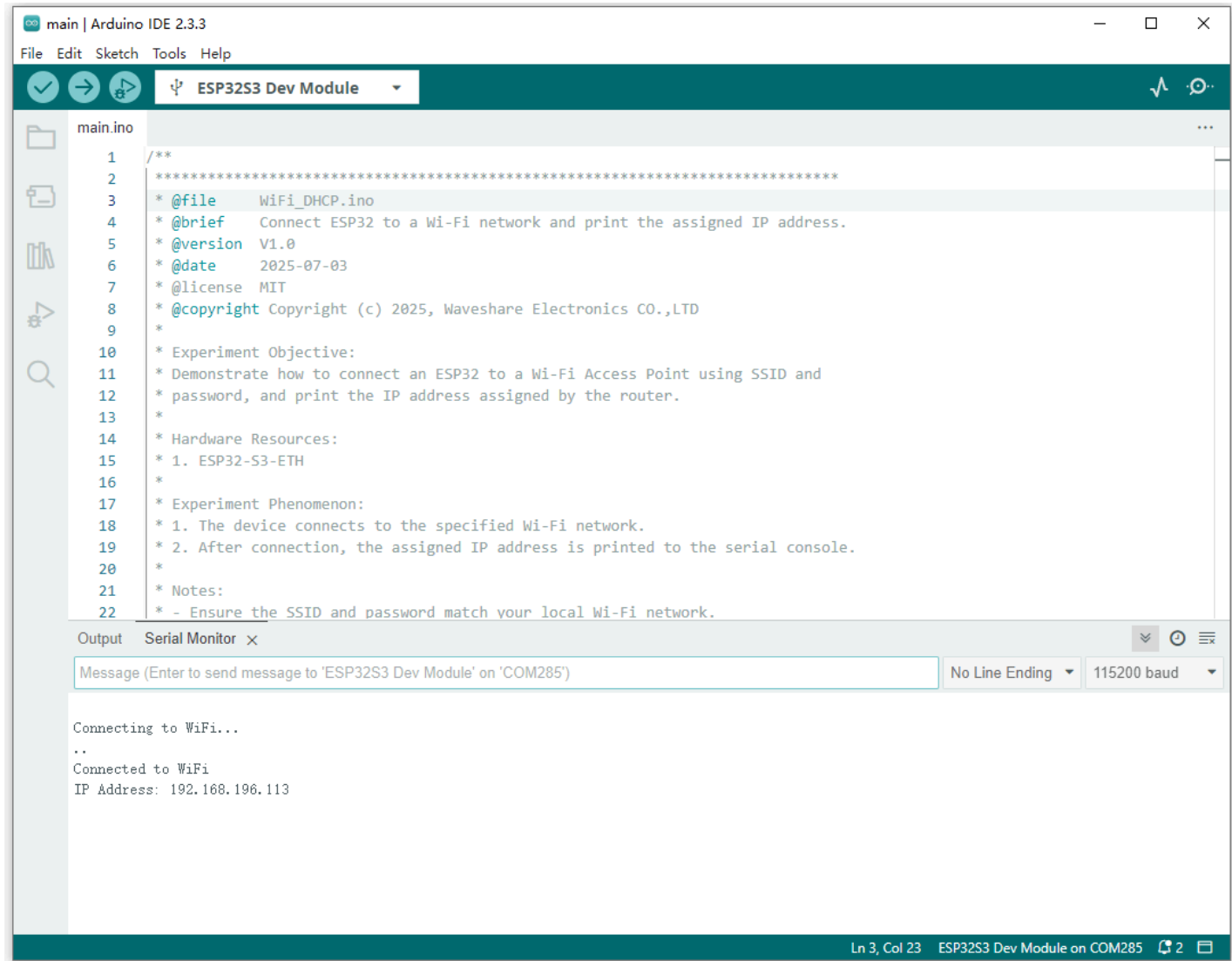
```
const char *ssid = "Waveshare";  
const char *password = "88888888";
```

2. setup()

- **Function:** Initialize system settings
- **Main steps:**
 - Initialize serial port communication (for debugging output).
 - Print the WIFI connection status and connect to the WIFI network according to the network configuration.
 - After connecting to WIFI successfully, print the connection success information and IP address.

Result demonstration

After startup, the serial port prints the status of the WIFI connection and the assigned IP.



The screenshot shows the Arduino IDE 2.3.3 interface. The top menu bar includes File, Edit, Sketch, Tools, and Help. The toolbar shows icons for checking, running, and uploading code, along with a dropdown menu for the board, currently set to 'ESP32S3 Dev Module'. The left sidebar contains icons for file explorer, search, and other IDE functions. The main editor displays the 'main.ino' file with the following code:

```
1  /**
2  *
3  * @file    WiFi_DHCP.ino
4  * @brief   Connect ESP32 to a Wi-Fi network and print the assigned IP address.
5  * @version V1.0
6  * @date    2025-07-03
7  * @license MIT
8  * @copyright Copyright (c) 2025, Waveshare Electronics CO.,LTD
9  *
10 * Experiment Objective:
11 * Demonstrate how to connect an ESP32 to a Wi-Fi Access Point using SSID and
12 * password, and print the IP address assigned by the router.
13 *
14 * Hardware Resources:
15 * 1. ESP32-S3-ETH
16 *
17 * Experiment Phenomenon:
18 * 1. The device connects to the specified Wi-Fi network.
19 * 2. After connection, the assigned IP address is printed to the serial console.
20 *
21 * Notes:
22 * - Ensure the SSID and password match your local Wi-Fi network.
```

Below the code editor is the 'Serial Monitor' window, which is currently empty. The status bar at the bottom indicates 'Ln 3, Col 23' and 'ESP32S3 Dev Module on COM285'.

(/wiki/File:ESP32-S3-ETH-Demo-WIFI-DHCP-RUN.png)

WIFI_StaticIP

[Collapse]

Demo description

The demo connects via WIFI using the **ESP32-S3-ETH module** and statically assigns an IP address.

Code analysis

1. **WIFI network configuration:** You need to change the WIFI and password in the code to the name and password in the WIFI router in your environment. If there is none, you can create a hotspot with the same name using your phone for module connection testing.

```
const char *ssid = "Waveshare";  
const char *password = "88888888";
```

2. Fixed IP configuration

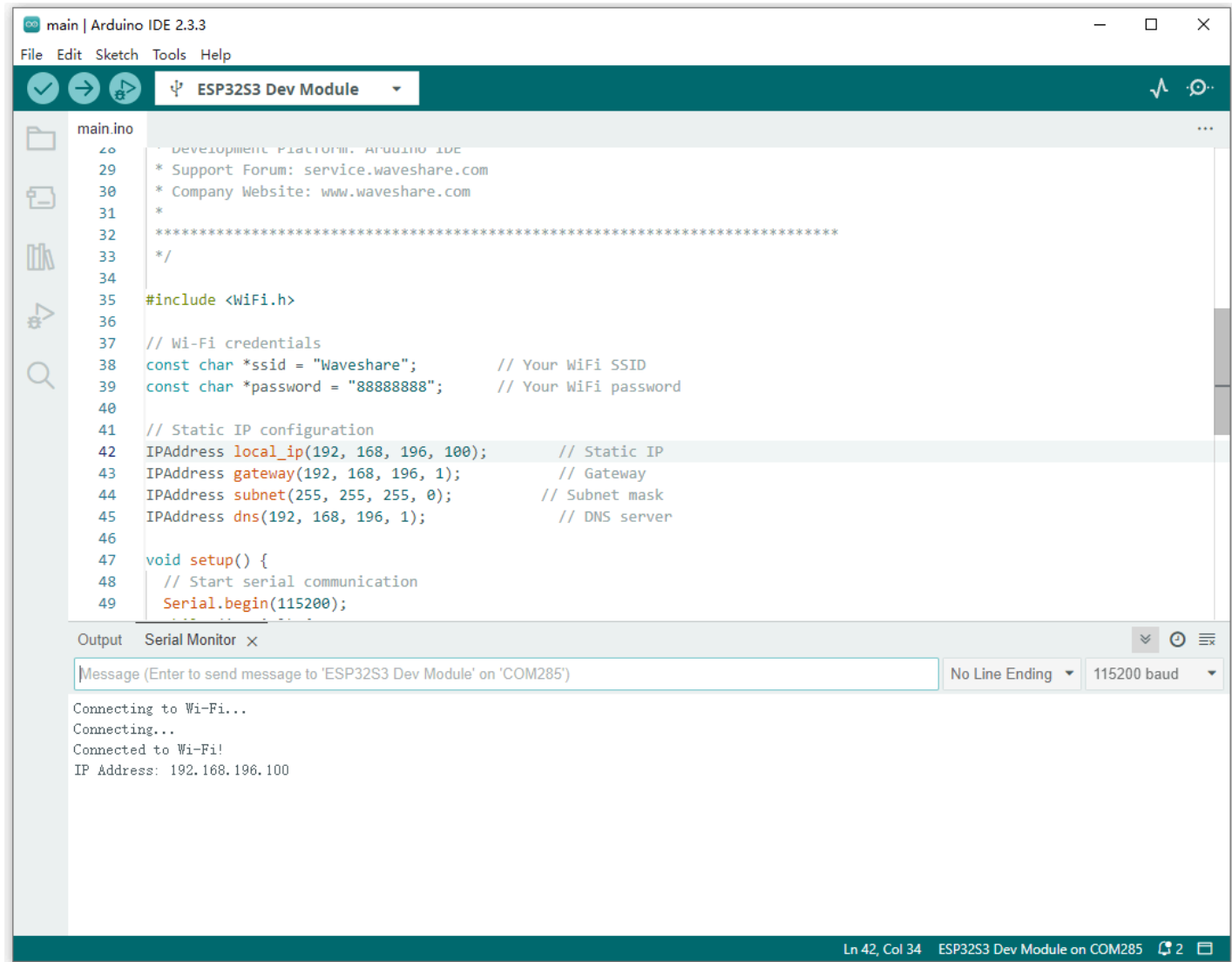
```
IPAddress local_ip(192, 168, 1, 100);    // Fixed IP address  
IPAddress gateway(192, 168, 1, 1);      // Gateway address  
IPAddress subnet(255, 255, 255, 0);    // Subnet mask  
IPAddress dns(192, 168, 1, 1);         // DNS server address
```

3. setup()

- **Function:** Initialize system settings
- **Main steps:**
 - Initialize serial port communication (for debugging output).
 - Set to LAN mode and configure a static IP address
 - Connect to the WIFI network according to the network configuration.
 - Print successful connection information and IP address.

Result demonstration

After startup, the serial port prints the status of the WIFI connection and the assigned IP.



The screenshot displays the Arduino IDE 2.3.3 interface. The top menu bar includes File, Edit, Sketch, Tools, and Help. The toolbar shows icons for checking, running, and uploading code, along with a dropdown menu for the ESP32S3 Dev Module. The left sidebar contains icons for file explorer, search, and other IDE functions. The main editor window shows the code for main.ino, which includes comments for the development platform, support forum, and company website. The code defines Wi-Fi credentials (ssid and password) and static IP configuration (local_ip, gateway, subnet, and dns). The setup function starts serial communication at 115200 baud. The serial monitor at the bottom shows the output: "Connecting to Wi-Fi...", "Connecting...", "Connected to Wi-Fi!", and "IP Address: 192.168.196.100". The status bar at the bottom indicates the current line and column (Ln 42, Col 34) and the selected module (ESP32S3 Dev Module on COM285).

```
main.ino
28  * Development Platform: ARDUINO IDE
29  * Support Forum: service.waveshare.com
30  * Company Website: www.waveshare.com
31  *
32  *****
33  */
34
35 #include <WiFi.h>
36
37 // Wi-Fi credentials
38 const char *ssid = "Waveshare";      // Your WiFi SSID
39 const char *password = "88888888";   // Your WiFi password
40
41 // Static IP configuration
42 IPAddress local_ip(192, 168, 196, 100); // Static IP
43 IPAddress gateway(192, 168, 196, 1);    // Gateway
44 IPAddress subnet(255, 255, 255, 0);    // Subnet mask
45 IPAddress dns(192, 168, 196, 1);       // DNS server
46
47 void setup() {
48   // Start serial communication
49   Serial.begin(115200);
50 }
```

Output Serial Monitor x

Message (Enter to send message to 'ESP32S3 Dev Module' on 'COM285') No Line Ending 115200 baud

Connecting to Wi-Fi...
Connecting...
Connected to Wi-Fi!
IP Address: 192.168.196.100

Ln 42, Col 34 ESP32S3 Dev Module on COM285 2

(/wiki/File:ESP32-S3-ETH-Demo-WIFI-StaticIP-RUN.png)

WIFI_Web_CAM

[\[Collapse\]](#)

Demo description

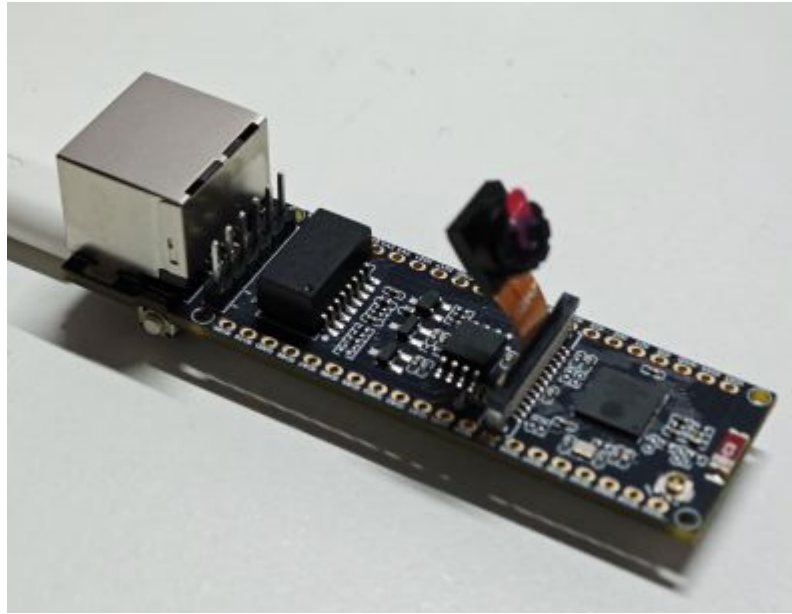
The demo uses the **ESP32-S3-ETH module** to connect via WIFI and combines with the camera module to achieve image acquisition and streaming transmission. The demo will configure the camera pins and start the camera server, allowing users to access the camera's content in real time through the WIFI LAN. During initialization process, the demo will set various parameters of the camera, including resolution, pixel format, etc.

Hardware connection

- Refer to the figure below to connect the hardware. The camera is connected using the DVP interface, and the pin definitions are shown in the diagram on the right.

Camera control pin description

Camera interface	ESP32-S3-ETH GPIO
VSYNC	GPIO1
HREF	GPIO2
XCLK	GPIO3
PCLK	GPIO39
SIOD (SDA)	GPIO48
SIOC (SCL)	GPIO47
D7	GPIO18
D6	GPIO15
D5	GPIO38
D4	GPIO40
D3	GPIO42
D2	GPIO46
D1	GPIO45
D0	GPIO41



(/wiki/File:400px-ESP32-S3-ETH-WIFI-

WEB-CAM.jpg)

Code analysis

1. **WIFI network configuration:** You need to change the WIFI and password in the code to the name and password in the WIFI router in your environment. If there is none, you can create a hotspot with the same name using your phone for module connection testing.

```
const char *ssid = "Waveshare";  
const char *password = "88888888";
```

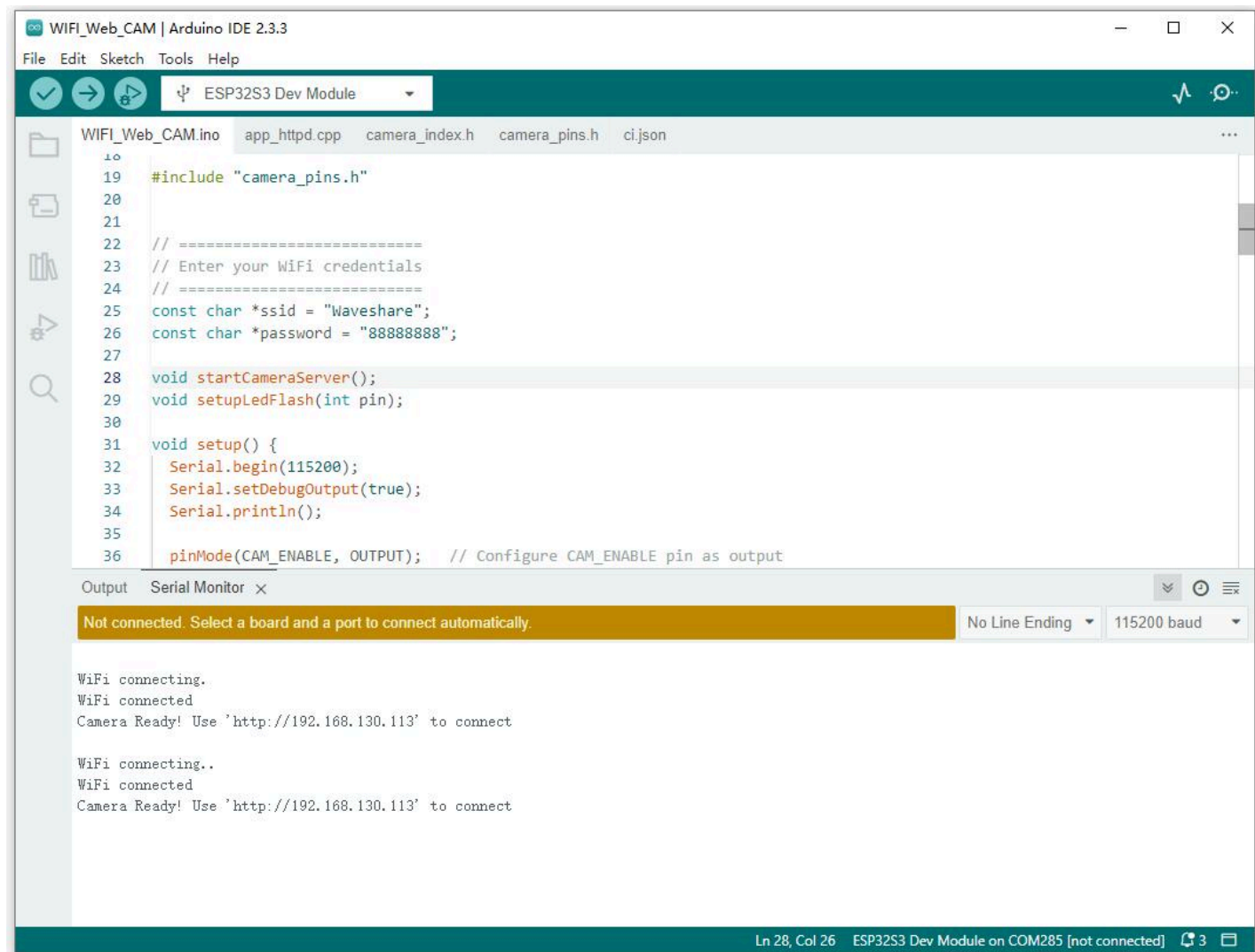
2. **startCameraServer()**

- It is used to start a web-based camera video streaming server. This function creates an HTTP server that receives client requests and returns images or video streams captured by the camera, allowing the user to access the camera data over the network.

How to know more about the CameraWebServer feature of the Arduino ESP32 library? Please refer to Arduino ESP32 library CameraWebServer use (<https://github.com/espressif/arduino-esp32/tree/master/libraries/ESP32/examples/Camera/CameraWebServer>).

Result demonstration

1. After startup, ESP32-S3 will initialize WIFI and output the connection status.
2. After the camera is successfully initialized, the server will start to provide real-time image transmission services, and users can connect to the camera server via WIFI to view real-time video.
3. If the connection is disconnected or the camera initialization fails, the demo will output an error message on the serial monitor.



The screenshot displays the Arduino IDE 2.3.3 environment. The main window shows the 'WIFI_Web_CAM.ino' file with the following code:

```
18
19 #include "camera_pins.h"
20
21
22 // =====
23 // Enter your WiFi credentials
24 // =====
25 const char *ssid = "Waveshare";
26 const char *password = "88888888";
27
28 void startCameraServer();
29 void setupLedFlash(int pin);
30
31 void setup() {
32   Serial.begin(115200);
33   Serial.setDebugOutput(true);
34   Serial.println();
35
36   pinMode(CAM_ENABLE, OUTPUT); // Configure CAM_ENABLE pin as output
```

The Serial Monitor at the bottom shows the output of the program:

```
WiFi connecting.
WiFi connected
Camera Ready! Use 'http://192.168.130.113' to connect

WiFi connecting..
WiFi connected
Camera Ready! Use 'http://192.168.130.113' to connect
```

The status bar at the bottom indicates the board is 'ESP32S3 Dev Module on COM285 [not connected]'.

(/wiki/File:ESP32-S3-ETH-WIFI-WEB-CAM-TEST01.jpg)



(/wiki/File:ESP32-S3-ETH-WIFI-WEB-CAM-TEST02.png)

Usage scenarios: Suitable for implementing low-cost network camera applications based on ESP32-S3, remote image acquisition applications, or IoT application scenarios such as monitoring, image recognition, and smart home.

ETH_DHCP

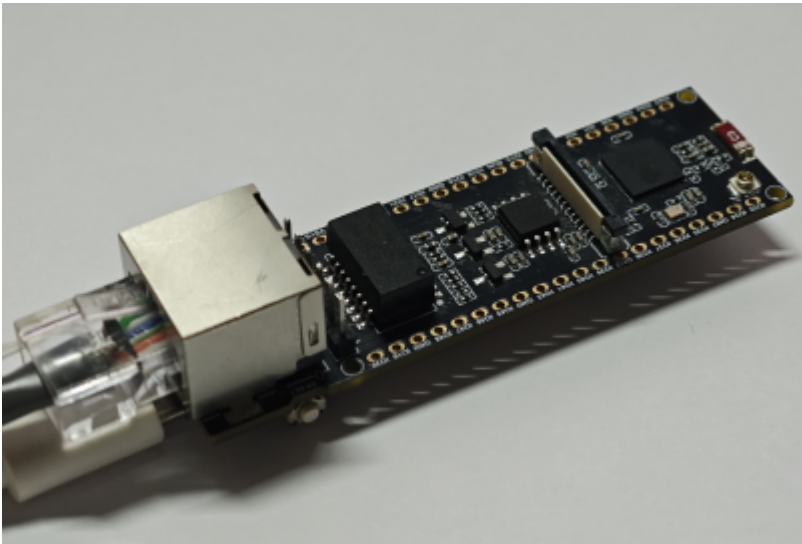
[Collapse]

Demo description

The demo connects via Ethernet using the **ESP32-S3-ETH module** and assigns an IP address through DHCP.

Hardware connection

- Connect the hardware according to the following diagram, as shown in the figure below



(/wiki/File:400px-ESP32-S3-ETH-ETH-USB-hw.png)

Network port control pin description

Network port interface	ESP32-S3-ETH GPIO
MISO	GPIO12
MOSI	GPIO11
SCLK	GPIO13
CS	GPIO14
RST	GPIO9
INT	GPIO10

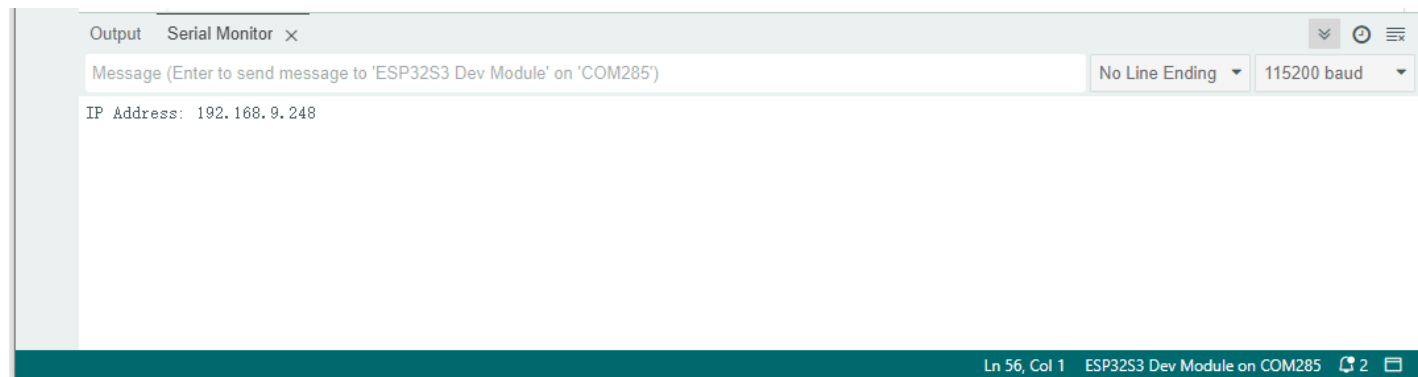
Code analysis

1. setup()

- **Function:** Initialize system settings
- **Main steps:**
 - Initialize serial port communication (for debugging output).
 - Initialize the SPI bus and configure the pins of the Ethernet communication module.
 - Initialize the Ethernet module and use DHCP to obtain an IP address. If it fails, stop the demo and continue if it succeeds.
 - Print the obtained IP address.

Result demonstration

After startup, ESP32-S3 will initialize Ethernet and print the IP address assigned by DHCP.



(/wiki/File:ESP32-S3-ETH-Demo-ETH-DHCP-RUN2.png)

ETH_StaticIP

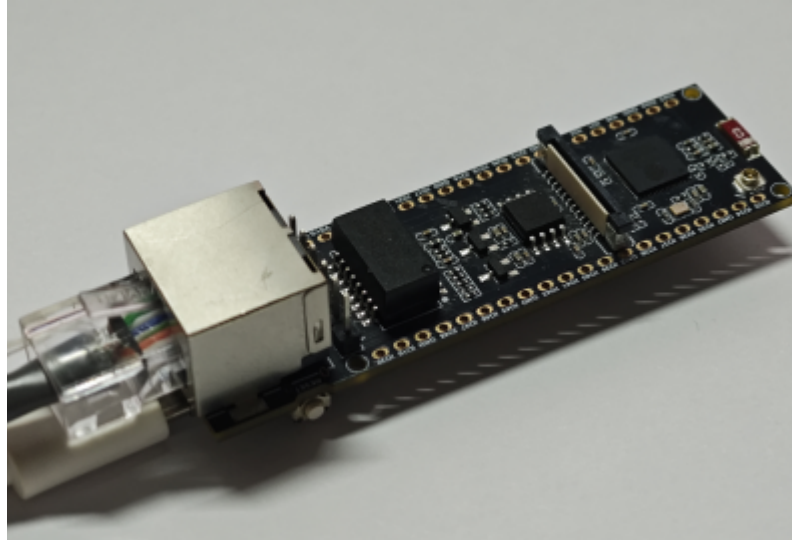
[\[Collapse\]](#)

Demo description

The demo connects via Ethernet using the **ESP32-S3-ETH module** and statically assigns an IP address.

Hardware connection

- Connect the hardware according to the following diagram, as shown in the figure below



(/wiki/File:400px-ESP32-S3-ETH-ETH-USB-

hw.png)

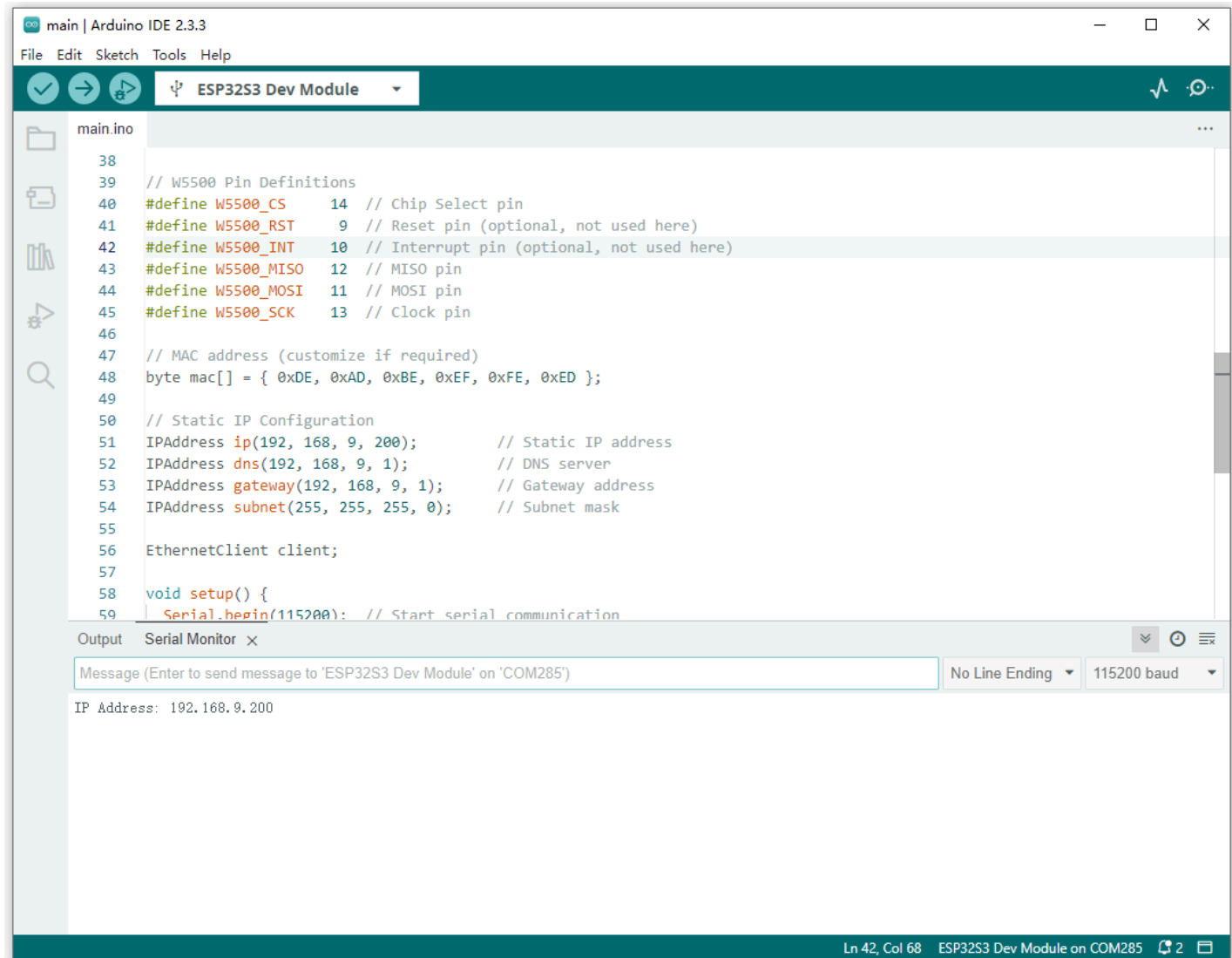
Code analysis

1. setup()

- **Function:** Initialize system settings
- **Main steps:**
 - Initialize serial port communication (for debugging output).
 - Initialize the SPI bus and configure the pins of the Ethernet communication module.
 - Initialize the Ethernet module and perform static IP configuration. If the IP address configuration fails, stop the demo, and continue if it succeeds
 - Print the obtained IP address.

Result demonstration

After startup, ESP32-S3 will initialize Ethernet and print the statically assigned IP address.



The screenshot displays the Arduino IDE 2.3.3 interface. The top menu bar includes File, Edit, Sketch, Tools, and Help. The toolbar shows icons for checking, running, and uploading code, along with a dropdown menu for the ESP32S3 Dev Module. The main editor window shows the main.ino file with the following code:

```
38
39 // W5500 Pin Definitions
40 #define W5500_CS 14 // Chip Select pin
41 #define W5500_RST 9 // Reset pin (optional, not used here)
42 #define W5500_INT 10 // Interrupt pin (optional, not used here)
43 #define W5500_MISO 12 // MISO pin
44 #define W5500_MOSI 11 // MOSI pin
45 #define W5500_SCK 13 // Clock pin
46
47 // MAC address (customize if required)
48 byte mac[] = { 0xDE, 0xAD, 0xBE, 0xEF, 0xFE, 0xED };
49
50 // Static IP Configuration
51 IPAddress ip(192, 168, 9, 200); // Static IP address
52 IPAddress dns(192, 168, 9, 1); // DNS server
53 IPAddress gateway(192, 168, 9, 1); // Gateway address
54 IPAddress subnet(255, 255, 255, 0); // Subnet mask
55
56 EthernetClient client;
57
58 void setup() {
59   Serial.begin(115200); // Start serial communication
```

The Serial Monitor window at the bottom shows the output: "IP Address: 192.168.9.200". The status bar at the bottom indicates "Ln 42, Col 68 ESP32S3 Dev Module on COM285".

(/wiki/File:ESP32-S3-ETH-Demo-ETH-StaticIP-RUN.png)

ETH_MAC_DHCP_StaticIP

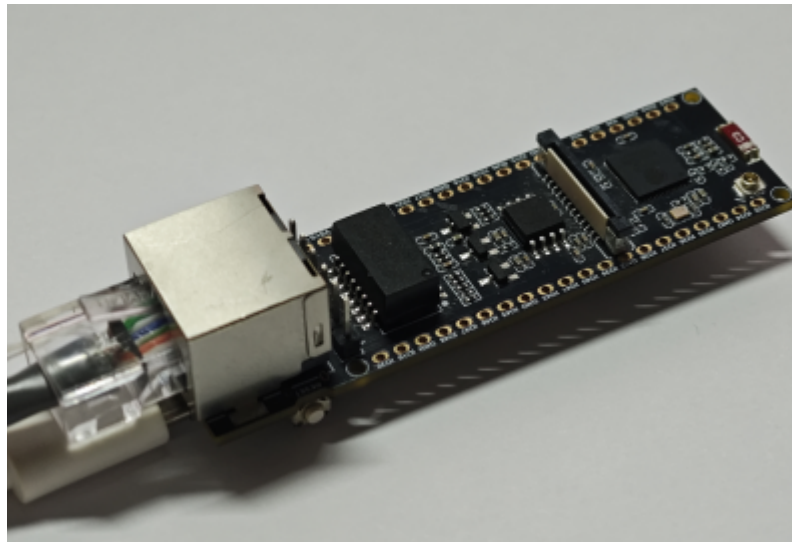
[\[Collapse\]](#)

Demo description

The demo automatically obtains the IP address via DHCP using the ESP32-S3-ETH module; If the DHCP acquisition fails, it falls back to using a preset static IP address. Subsequently, the final assigned IP address is printed to the serial console for debugging and network verification.

Hardware connection

- Connect the hardware according to the following diagram, as shown in the figure below



(/wiki/File:400px-ESP32-S3-ETH-ETH-USB-

hw.png)

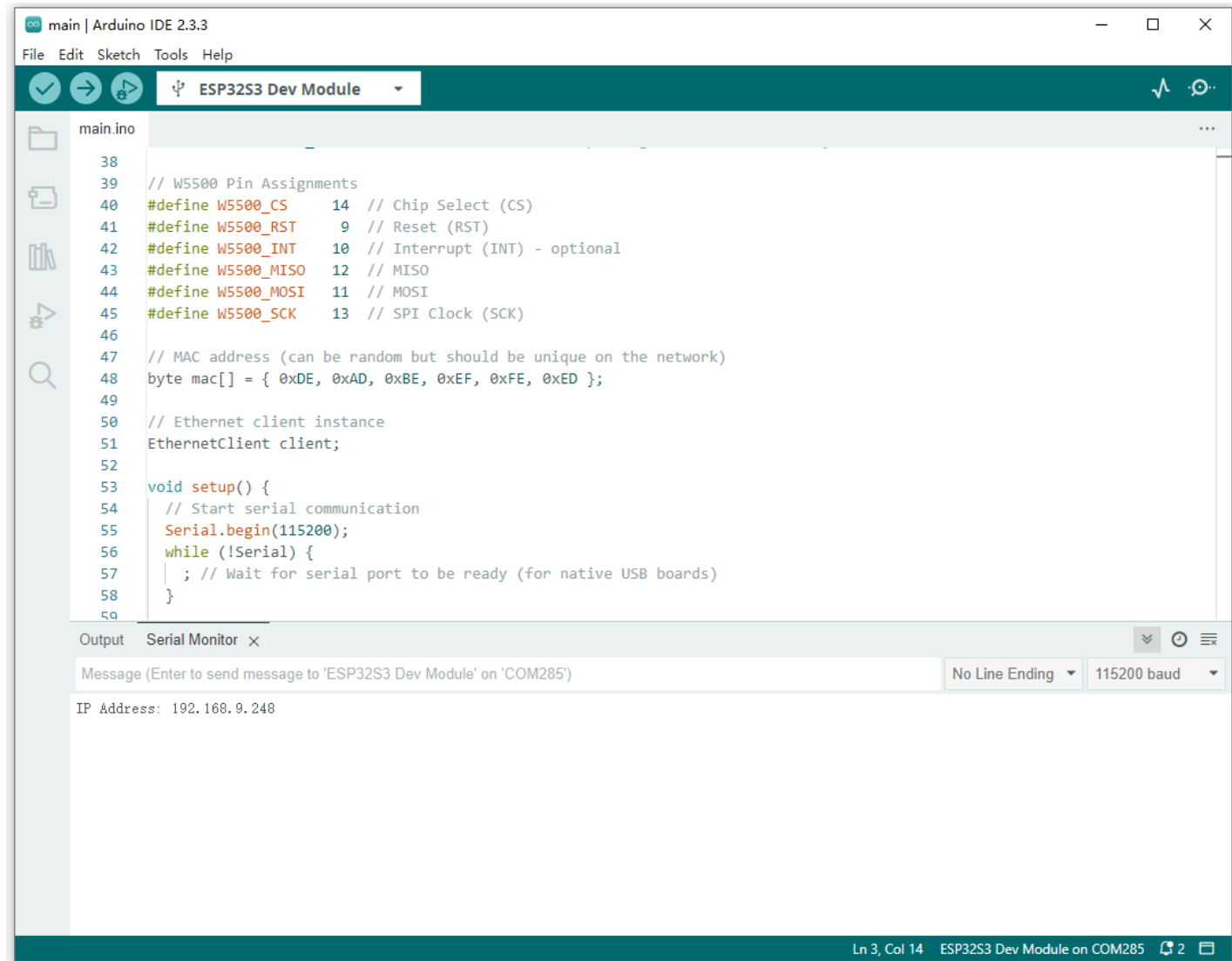
Code analysis

1. setup()

- **Function:** Initialize system settings
- **Main steps:**
 - Initialize serial port communication (for debugging output).
 - Initialize the SPI bus and configure the pins of the Ethernet communication module.
 - Initialize the Ethernet module, first attempt to obtain an IP address via DHCP; if it fails, use the user-preset static IP address (e.g., 192.168.1.177) for initialization.
 - Print the obtained IP address.

Result demonstration

After startup, ESP32-S3 will initialize Ethernet and print the statically assigned IP address.



```
main | Arduino IDE 2.3.3
File Edit Sketch Tools Help
ESP32S3 Dev Module
main.ino
38
39 // W5500 Pin Assignments
40 #define W5500_CS 14 // Chip Select (CS)
41 #define W5500_RST 9 // Reset (RST)
42 #define W5500_INT 10 // Interrupt (INT) - optional
43 #define W5500_MISO 12 // MISO
44 #define W5500_MOSI 11 // MOSI
45 #define W5500_SCK 13 // SPI Clock (SCK)
46
47 // MAC address (can be random but should be unique on the network)
48 byte mac[] = { 0xDE, 0xAD, 0xBE, 0xEF, 0xFE, 0xED };
49
50 // Ethernet client instance
51 EthernetClient client;
52
53 void setup() {
54   // Start serial communication
55   Serial.begin(115200);
56   while (!Serial) {
57     ; // Wait for serial port to be ready (for native USB boards)
58   }
59 }
```

Output Serial Monitor x

Message (Enter to send message to 'ESP32S3 Dev Module' on 'COM285') No Line Ending 115200 baud

IP Address: 192.168.9.248

Ln 3, Col 14 ESP32S3 Dev Module on COM285 2

(/wiki/File:ESP32-S3-ETH-Demo-ETH-DHCP-StaticIP-RUN.png)

ETH_Web_CAM

[Collapse]

Demo description

The demo utilizes the **ESP32-S3-ETH module** for Ethernet connectivity and, in conjunction with a camera module, implements image acquisition and streaming. The demo will configure the camera pins and start the camera server, so that the user can access the camera content in real time via Ethernet. During initialization, the demo sets various parameters of the camera, including resolution, pixel format, etc., and monitors the status of the Ethernet connection through network events.

Hardware connection

- Refer to the figure below to connect the hardware (if you don't need PoE power supply, there is no need to connect the PoE module)

Camera control pin description

Camera interface	ESP32-S3-ETH GPIO
VSYNC	GPIO1
HREF	GPIO2
XCLK	GPIO3
PCLK	GPIO39
SIOD (SDA)	GPIO48
SIOC (SCL)	GPIO47
D7	GPIO18
D6	GPIO15
D5	GPIO38
D4	GPIO40
D3	GPIO42
D2	GPIO46
D1	GPIO45
D0	GPIO41



(/wiki/File:400px-ESP32-S3-ETH-details-12.jpg)

Network port control pin description

Network port interface	ESP32-S3-ETH GPIO
MISO	GPIO12
MOSI	GPIO11
SCLK	GPIO13
CS	GPIO14
RST	GPIO9
INT	GPIO10

Code analysis

1. setup()

- **Function:** Configure serial port debugging and initialize the Ethernet connection, and set the camera control pins.
- Start Ethernet according to different ESP32 configurations through conditional compilation:
 - **SPI mode:** On non-ESP32 devices, use "ETH.beginSPI()" to start Ethernet.
- After waiting for the Ethernet connection to be successful, configure the camera parameters and start the camera server for external access.
- **Camera configuration:** Configure various parameters of the camera through the "camera_config_t" structure, including:
 - Image resolution, format (JPEG or RGB565), XCLK frequency, frame buffer, etc.
 - If PSRAM is detected, higher JPEG quality and frame buffer settings will be used to improve image quality.

2. startCameraServer()

- Start the camera server so that the user can access the camera content through the network.

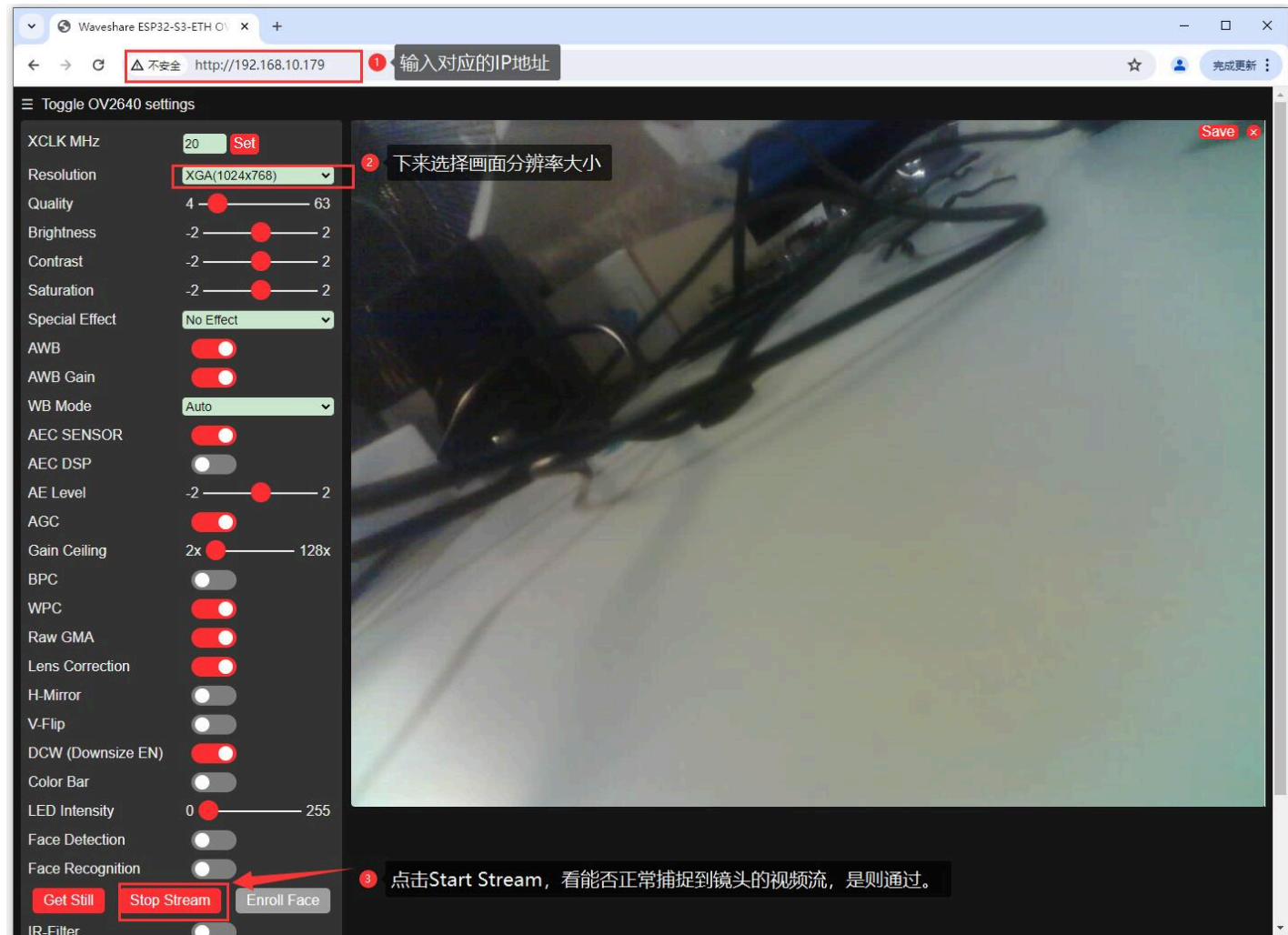
3. Event handling function `WiFiEvent(WiFiEvent_t event)`

- **Function:** Used to monitor the status of the Ethernet connection and output debugging information when an event occurs.
 - **Connection event :** When Ethernet is started, connected, or disconnected, status information is output.
 - **Get IP address event:** When an IP address is successfully obtained, the MAC address, IPv4 address, full-duplex status, link speed, and gateway IP address of the device are printed.

How to know more about the CameraWebServer feature of the Arduino ESP32 library? Please refer to Arduino ESP32 library CameraWebServer use (<https://github.com/espressif/arduino-esp32/tree/master/libraries/ESP32/examples/Camera/CameraWebServer>).

Result demonstration

1. After startup, the ESP32-S3 initializes the Ethernet and outputs the connection status.
2. After the camera is successfully initialized, the server will start to provide real-time image transmission services, and users can connect to the camera server through Ethernet to view real-time video.
3. If the connection is disconnected or the camera initialization fails, the demo will output an error message on the serial monitor.



(/wiki/File:ESP32-S3-ETH-Demo-ETH_WEB_CAM-running.jpg)

Usage scenarios: Suitable for implementing low-cost network camera applications based on ESP32-S3, remote image acquisition applications, or IoT application scenarios such as monitoring, image recognition, and smart home.

Flash Firmware Flashing and Erasing

- **The current demo provides test firmware, which can be used to test whether the onboard device functions properly by directly flashing the test firmware**

Flash firmware flashing and erasing (https://www.waveshare.com/wiki/Flash_Firmware_Flashing_and_Erasing) for reference

Resources

Schematic Diagram

- ESP32-S3-ETH Schematic (<https://files.waveshare.com/wiki/ESP32-S3-ETH/ESP32-S3-ETH-Schematic.pdf>)

Demo

- ESP32-S3-ETH Demo (with library files) (<https://files.waveshare.com/wiki/ESP32-S3-ETH/ESP32-S3-ETH-Demo.zip>)

Datasheets

ESP32-S3

- ESP32-S3 Datasheet (https://files.waveshare.com/wiki/common/Esp32-s3_datasheet_en.pdf)

- ESP32-S3 Technical reference manual (https://files.waveshare.com/wiki/common/Esp32-s3_technical_reference_manual_en.pdf)

Other Components

- W5500 Datasheet (https://files.waveshare.com/wiki/common/W5500_ds_v110e.pdf)

Software Tools

Arduino

- Arduino IDE Official download link (<https://www.arduino.cc/en/software/>)
- ESP32_Arduino offline package (https://drive.google.com/drive/folders/1Pcs_A4FKWvdSHnz9lEBYqOpr-noTMbIv?usp=sharing)

Debugging Tools

- SSCOM debugging assistant (<https://files.waveshare.com/wiki/common/Sscom5.13.1.zip>)

Other Resource Links

- ESP32-Arduino official documentation (<https://docs.espressif.com/projects/arduino-esp32/en/latest/index.html>)

Project Resources

This section features third - party project resources. We merely provide links and bear no responsibility for content updates or maintenance. Thank you for your understanding.

DroneBot Workshop-Ethernet & Power-over-Ethernet with ESP32 - Build a PoE Camera and a Web Server



Ethernet & Power-over-Ethernet with E!
DroneBot Workshop



Watch on

- Youtube: <https://www.youtube.com/watch?v=JgwjGeAqW8E> (<https://www.youtube.com/watch?v=JgwjGeAqW8E>)
- Guide: <https://dronebotworkshop.com/esp32-ethernet/> (<https://dronebotworkshop.com/esp32-ethernet/>)

FAQ

Question: After the module downloads the demo and re-downloads it, why sometimes it can't connect to the serial port or the flashing fails?

Answer:

Long press the BOOT button, press RESET at the same time, then release RESET, then release the BOOT button, at this time the module can enter the download mode, which can solve most of the problems that can not be downloaded.

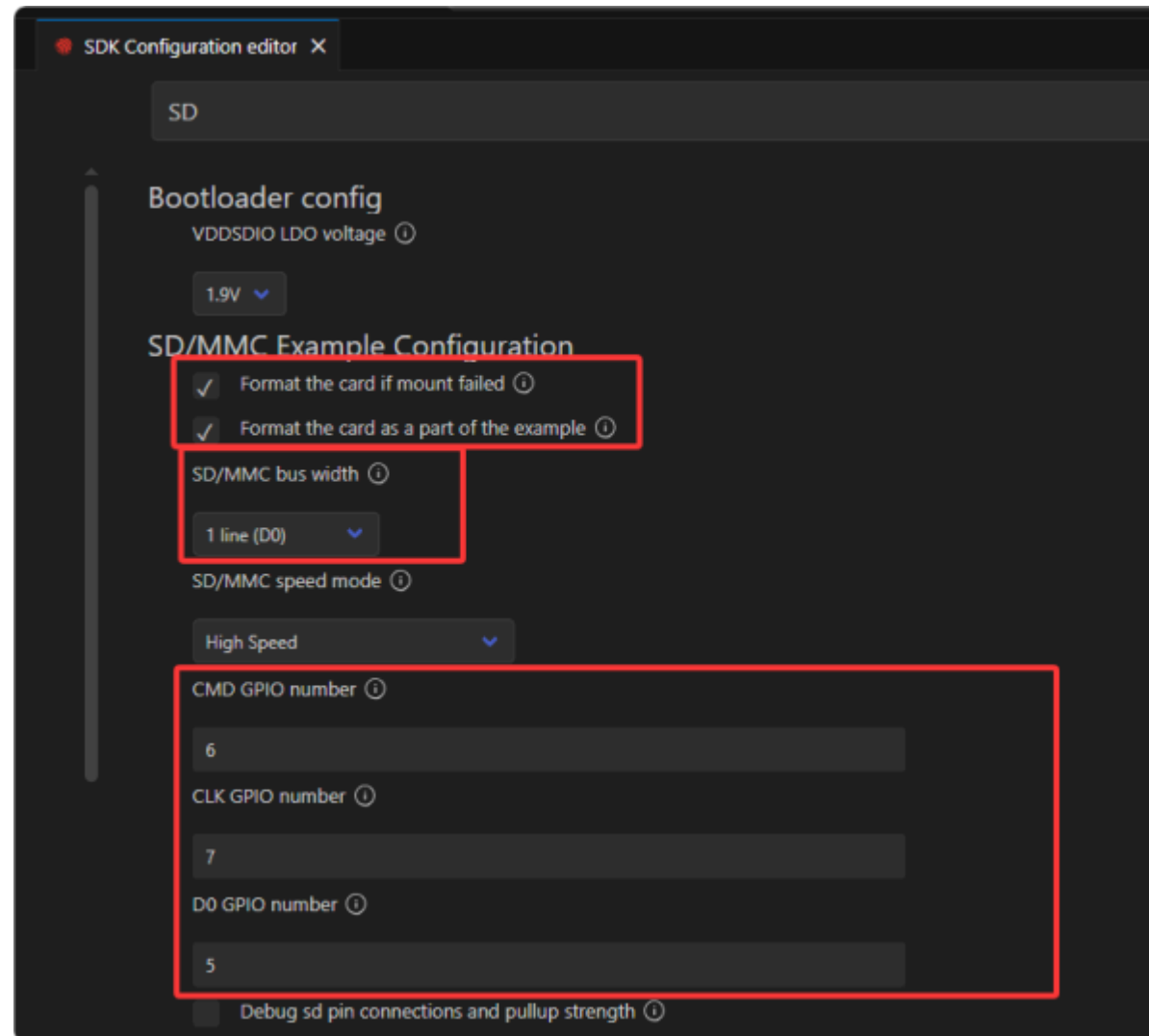
Question: Which IOs are unavailable on the development board?

Answer:

GPIO33~GPIO37 are internally occupied and cannot be used.

Question: Does the development board support SDIO access to TF cards?**Answer:**

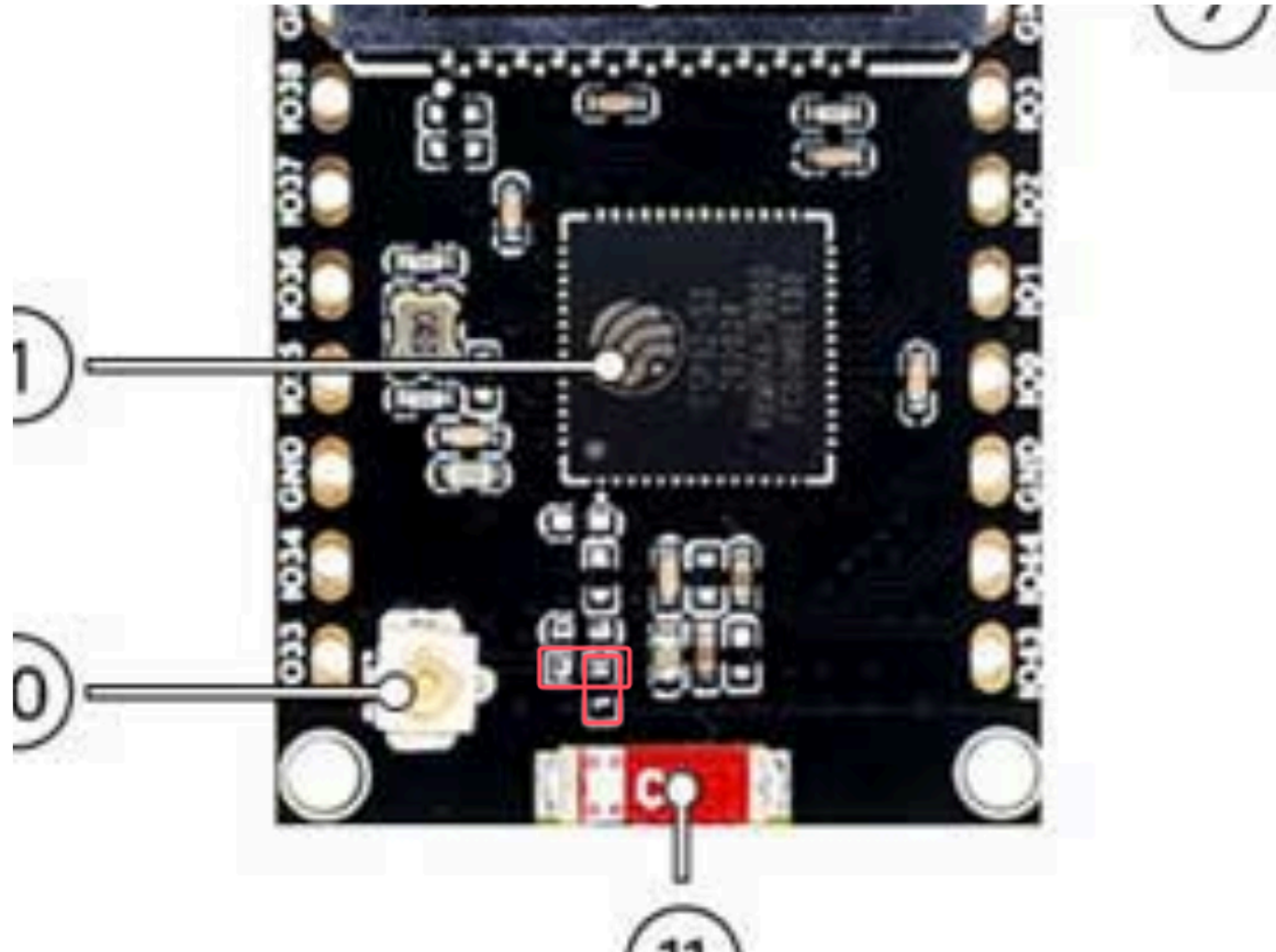
Yes, but only in 1-line SD mode. Pins need to be configured in menuconfig, as shown below:



(/wiki/File:ESP32-S3-ETH-01.png)

Question: How to switch to the external antenna for IPEX 1 generation?**Answer:**

As shown in the figure below, the default weld is a vertical 0R resistor. If you want to switch to an external antenna, you need to re-solder the 0R resistor to a horizontal position:



(/wiki/File:ESP32-S3-ETH-Antena-Switch.png)

Question: Espressif official FAQ summary

Answer:

<https://docs.espressif.com/projects/esp-faq/en/latest/index.html> (<https://docs.espressif.com/projects/esp-faq/en/latest/index.html>)

Support

Technical Support

If you need technical support or have any feedback/review, please click the **Submit Now** button to submit a ticket, Our support team will check and reply to you within 1 to 2 working days. Please be patient as we make every effort to help you to resolve

Submit Now (<https://service.waveshare.com/>)

the issue.

Working Time: 9 AM - 6 PM GMT+8 (Monday to
Friday)

Retrieved from "<https://www.waveshare.com/w/index.php?title=ESP32-S3-ETH&oldid=110639> (<https://www.waveshare.com/w/index.php?title=ESP32-S3-ETH&oldid=110639>)"
